论文草稿 v0.9

题目：当记忆库压缩改变工业报警：决策保持的边缘筛查及其在外观变化下的代价

英文：When Memory Compression Changes Industrial Alarms: Decision-Preserving Edge Screening and Its Cost under Appearance Shift

作者：吴源泰 ｜ **日期**：2026-10-11 ｜ **状态**：草稿，数据与代码已冻结（commit `2f80ea4`）

摘要

工业异常检测的公开指标已经饱和，但部署中真正决定系统可用性的是一个固定阈值下的**报警决策**是否稳定。我们研究一个此前未被单独度量的现象：**把正常参考库（记忆库）压缩之后，同一个冻结检测器、同一个阈值，会不会改变报警**。我们在 MVTec AD 的三个类别（bottle / tile / screw）上实测：压缩到 1% 时，与全库相比的二元分歧达 **24.1% / 12.0% / 41.9%**，而且**全部是多报、没有一条漏报**；在 5–10% 的"有用存储压缩"区间仍有 **2–4 个百分点且 2/3 类别超出门槛**的分歧。用压缩库单独重校准可以把残余压到 0–2.5%，但消不掉近阈值那部分。

随后我们检验一种**可证明保持决策**的边缘筛查方案：只下发 cell 观测中心与保守半径，用三角不等式给出分数区间 `[L,U]`，落在区间外的图直接裁决，横跨阈值的图升级查询整库。该方案在 **720 条决策上零错判**，本地裁决比例 **10.8%–89.2%**（随类别与粒度变化）。我们还给出代价结构：粗粒度索引只有 192 KB（全库的 0.26%），但几乎每张图都要升级；细粒度 7.5 MB 的升级率降到 10.8–61.5%，只有在**远端解析器（传输 ≥20 ms）**时才比每次查整库更省。此外我们报告一个负面结果：**精确树在 1536 维下比 GPU 暴力慢约 1000 倍**，因此"用精确树当更省的替代"不是正确的对照设计。

关键词：异常检测；记忆库压缩；报警决策保持；边缘筛查；距离区间；MVTec AD

1 引言

异常检测的评测长期以 AUROC / AU-PRO 为主，而部署中真正被运维关心的只有两件事：**这次该不该报警**，以及**边缘端承担得起多少存储与查询**。这两件事之间几乎没有人做定量连接。

本项目的研究对象是一个很具体的偏差：**压缩会改变分数分布，因而改变固定阈值下的报警**。我们不去提出新的最近邻定理，而是提出三个可证伪的实证问题：

（i）在什么压缩比下、哪些类别的决策会变、往哪个方向变；

（ii）用保守距离区间能否在不查整库的前提下把这些决策复核回来，代价多大；

（iii）外观变化时这个能力还在不在。

我们的贡献是：

1. **实测了压缩造成的决策分歧**，并揭示其方向性（全为多报）与对检测器余量的依赖；
2. **实现并审计了一个可证明保持决策的筛查器**（版本契约、未决处理、单级/两级半径），并在真实数据上做到零错判；
3. **量化了代价结构**，给出盈亏平衡点，并报告精确树在高维退化的负面结果；
4. **如实记录了复现偏离**（screw 差 2.3 个 AUROC 点）与两个实现陷阱（medoid、父中心保留），这些陷阱会静默破坏区间性质。

2 相关工作

- **PatchCore（R1）**：核心集记忆库 + 重加权最近邻。本文的检测器是它的"max-score 简化版"，不实现重加权，因为区间证明只覆盖 max 聚合。
- **AnomalyDINO（R2）**：DINOv2 特征 + top-tail 平均分数；同样不在本文证明范围内。

- **EfficientAD（R3）**：轻量学生-教师，用于成本对照（本文未实现，列为限制）。
- **MVTec AD（R4）/ VisA**：数据基准；AD 2 强调真实外观变化与私有测试服。
- **核心集与度量检索（R5, R6）**：贪心 k-中心；本文用它构造 cell。
- **三角不等式上界检索（R7）**：区间思想的标准来源，本文不主张其新颖性。
- **符合预测集（R8）** 与 **记忆污染（R9）**：与我们最接近的两个"可靠性"工作。区别：R8 度量覆盖与集合大小，R9 度量检索是否跨产品；**本文度量的是固定阈值下的二元决策是否与冻结整库一致，且未决时绝不静默降级**。

3 方法

3.1 记号

M 为参考库（patch 向量），q 为查询 patch，d 为欧氏距离。c_j ∈ M 为 cell 中心，C_j = {m : c_j = argmin_c d(m,c)}，r_j ≥ max_{m∈C_j} d(m,c_j)。

3.2 区间

```
L(q) = min_j max{0, d(q,c_j) − r_j}   ≤   s_M(q) = min_{m∈M} d(q,m)   ≤   U(q) = min_j d(q,c_j)
```

证明见附录 C。关键前提：**c_j 必须是 M 中的实际向量**（medoid），否则 U 不再是上界。

3.3 决策与未决

```
S_M(x) = max_q s_M(x,q)            （严格大于才报警）
U_x ≤ t → 正常 ；L_x > t → 报警 ；否则升级
解析器离线 / 版本不符 → 未决（绝不静默判正常）
```

3.4 两级细化

对大 cell 用 k-means 切子簇，子中心取 **medoid**，并**强制包含父中心**。叶子仍构成 M 的划分，故区间性质保持。实测区间宽度 5.696 → 5.270（随机数据）、升级率在 tile/screw 上显著下降（§5.4）。

3.5 版本契约

`BankVersion.id() = sha256(编码器，预处理，库哈希，聚合方式，阈值，K，压缩比，层数)`。

任一项变化即为新版本，半径与阈值必须重算；版本不符只能产生"未决"。

4 实验设置

- **数据**：MVTec AD（CC BY-NC-SA 4.0），bottle/tile/screw；15 类逐项计数校验通过（3629/1725）。

官方下载页 16 个链接全部 404，经镜像获取，归档 sha256 与镜像登记一致。

- **角色**：reference-fit 16 / development 24 / calibration 169–280，**互不重叠**（重叠 0）。
- **检测器**：冻结 WRN-50（layer2+layer3，1536 维，3×3 邻域聚合，224px），图像分数 = max patch 距离。
- **阈值**：校准正常图的 95 分位（名义 5% FAR），不读缺陷标签。
- **压缩**：全库的 1/5/10/25%（贪心 k-中心）。
- **度量**：二元分歧（拆多报/漏报）、FAR、召回、AUROC、本地裁决率、升级率、错判数、字节、时延。
- **复现**：全部脚本 + JSON + 图，`commit 2f80ea4`；单条命令可重跑。

5 结果

5.1 表 1：复现（全量参考库，只算 AUROC）

类别	库 patch	本文	论文	差
bottle	163,856	**1.0000**	100.0%	0.0
tile	180,320	**0.9809**	98.7%	−0.6
screw	250,880	**0.9582**	98.1%	**−2.3**

screw 超出 2 点容差。偏离项：无重加权、无随机投影、无 FAISS 精确保留、torch 2.6 vs 1.x。

5.2 表 2：压缩造成的决策分歧（16-shot，库 12,544 patch）

类别	1%	5%	10%	25%	重校准后 (10%)	升级率	本地错判
bottle	**24.10%**	3.61%	0.00%	0.00%	0.00%	10.8%	0
tile	**11.97%**	3.42%	2.56%	2.56%	1.71%	61.5%	0
screw	**41.88%**	7.50%	3.12%	0.00%	2.50%	15.6%	0

全部 720 条分歧均为多报，漏报 0。

5.3 表 3：字节、时延与盈亏平衡

类别	编码器	全库暴力	BallTree	筛查 k=32	k=128	k=1254
bottle	31.8 ms	9.02 ms	8963 ms	3.41 ms	4.51 ms	23.70 ms
tile	42.1 ms	8.38 ms	17352 ms	3.55 ms	4.48 ms	23.67 ms
screw	34.6 ms	11.61 ms	15350 ms	3.38 ms	4.42 ms	24.00 ms

字节：k=32 → 192.2 KB（0.26%） | k=128 → 768.5 KB（1.02%） | k=1254 → 7528.9 KB（10.00%）。

盈亏平衡 `p* = (T_transfer + T_resolver − T_screen)/(T_transfer + T_resolver)`：

K	T=0	20 ms	50 ms	100 ms
32	0.58–0.71	0.88–0.89	0.94	0.97
1254	0.00	0.17–0.24	0.60–0.61	0.78

对照实测升级率：k=1254 在 bottle/screw 上（10.8%/15.6%）低于 p*(20ms) 以外的点 → 划算；tile（61.5%）刚好不划算；k=32 全部不划算（升级 98–100%）。

5.4 表 4：两级细化消融（k=32 强制粗粒度）

类别	一级 本地率	二级 本地率	二级字节
bottle	1.2%	1.2%	576 KB
tile	1.7%	**26.5%**	504 KB
screw	0.0%	**9.4%**	552 KB

全局半径 vs 逐 cell 半径差异很小（tile 二级 26.5% vs 25.6%）。

5.5 表 5：参考数量与种子

类别	1-shot	4-shot	16-shot	40-shot	5 种子范围（16-shot）
bottle	78.0%	74.7%	89.2%	89.2%	89.2–90.4%
tile	48.7%	40.2%	38.5%	36.8%	38.5–40.2%
screw	94.4%	91.2%	84.4%	74.4%	82.5–85.6%

tile/screw 的本地裁决率随参考增加而下降；索引字节随库线性增长（468 KB → 18.8 MB）。

5.6 表 6：外观变化（受控合成扰动，非真实光照）

类别	变换	FAR	召回	升级率	决策翻转
bottle	identity	0.0%	100%	10.8%	0
bottle	bright+30%	5.0%	100%	10.8%	1
tile	identity	0.0%	84.5%	61.5%	0
tile	gamma0.7	0.0%	78.6%	55.6%	5
screw	identity	9.8%	11.8%	15.6%	0

类别	变换	FAR	召回	升级率	决策翻转
screw	bright+30%	12.2%	16.0%	18.1%	6

决策翻转在 83–160 张图里只有 0–10 张；**升级率基本稳定**。但这是光度扰动，不能替代 AD 2 的真实外观变化。

5.7 图

- **图 1**（`figure_5_real_disagreement`）：分歧 vs 压缩比，固定阈值与重校准两条曲线。
- **图 2**（`figure_6_real_tradeoff`）：分歧（压缩的代价）与升级率（筛查的代价）并列。
- **图 3**（`figure_7_shots_seeds`）：参考数量与种子的敏感性。
- **失败图库**：`gallery_{bottle,tile,screw}`，含真值掩码与逐图 L/U/s/t。

6 讨论

1. **为什么全是多报**：压缩抬高最近邻距离，分数整体右移，固定阈值下的决策单向往报警侧移动。漏报需要压缩降低距离，而这在贪心核心集下几乎不会发生。
2. **为什么 bottle 不敏感**：它的 AUROC=1.0，决策远离阈值，区间省下的宽度用不上。**分歧率是检测器余量的函数**，不是压缩的固有属性。
3. **为什么精确树崩了**：1536 维下树的上界剪枝几乎全失效，退化为逐点扫描且常数更大。这推翻了"树是更省的替代"这一常见假设。
4. **为什么两级只在部分类别有效**：细化的收益取决于半径是否真的是约束。tile 的区间最宽，细化收益最大；bottle 的区间本来就不是瓶颈。
5. **代价结论**：筛查省的是**带宽**（192 KB vs 7.5 MB），不是**算力**；只有远端解析器 + 传输足够慢时才有净收益。

7 局限与威胁

1. **screw 复现未达标**（−2.3 点），且 16-shot 下其召回仅 11.8%——用它谈"决策保持"证据力弱。
2. **FAR 的样本量小**（test/good 20–41 张），"0% FAR"与 5% 兼容，不能支撑低误报主张。
3. **位移是合成扰动**，不是 AD 2 的真实外观变化；AD 2 未获批。
4. **仅三个类别、单一种子用于主表**（种子方差另列）。
5. **传输时延为受控假设**，无真实边缘链路测量。
6. **筛查实现为 CPU numpy**，细粒度时延被高估，结论偏保守。
7. **未实现 EfficientAD / AnomalyDINO 对照**，成本比较缺一个强基线。

8 结论

压缩会改变报警，而且几乎只朝误报方向改；单靠重校准消不掉近阈值那部分。距离区间能在**零错判**的前提下把相当比例的决策留在本地，但收益高度依赖类别、粒度与链路时延——在本地快速解析器下，细粒度筛查反而不划算。我们认为这个"代价图景"本身比任何单点数字更有用：它告诉实践者在什么条件下该用筛查、什么条件下不如直接把库做小。

附录 A：主张-证据对照表

#	主张	证据	强度
C1	压缩改变固定阈值下的报警	表 2：1% 时 24.1/12.0/41.9%	强（3 类一致）

#	主张	证据	强度
C2	改变几乎全为多报	表 2：720 条分歧中漏报 0	强
C3	重校准不充分	表 2：残余 0–2.5% 集中在近阈值	中（样本小）
C4	区间筛查保持决策	表 2/4/5：全部条件错判 0；附录 C 证明	强（有证明 + 实测）
C5	升级代价高度依赖类别	表 2：10.8 / 61.5 / 15.6%	强
C6	两级细化在区间为约束时有效	表 4：tile +24.8 pp、screw +9.4 pp、bottle 0	中（k=32 特定）
C7	筛查只在远端解析器下净收益	表 3 盈亏平衡 + 实测升级率	中（传输为假设）
C8	精确树在高维退化	表 3：比 GPU 暴力慢约 1000×	强
C9	光度扰动下决策稳定	表 6：翻转 0–10 张	弱（合成、非真实位移）
C10	未决路径绝不留静默降级	端到端：断网/版本不符时 9/72/25 张全部未决	强（断言 + 实测）

附录 B：可复现清单

```
python scripts\featurize.py --categories bottle tile screw
python scripts\screening.py --selftest
python scripts\screening.py --category all --ratio 0.10 --levels 1 2 --radius cell global
python scripts\screening.py --category all --k 32 --levels 1 2 --radius cell global --out
logs\w4_screening_k32.json
python scripts\benchmark.py --category bottle --k 32 128 1254
python scripts\failure_gallery.py --category bottle
python scripts\shift_test.py --categories bottle tile screw
python scripts\make_figures_real.py ; python scripts\make_figures_phase7.py
```

附录 C：区间性质证明

上界：任意 j 有 $d(q,c_j) \geq s_M(q)$（$c_j \in M$），故 $U(q) = \min_j d(q,c_j) \geq s_M(q)$。

下界：取任意 $m \in M$ 属 cell j，三角不等式 $d(q,m) \geq d(q,c_j) - d(c_j,m) \geq d(q,c_j) - r_j$，又 $d \geq 0$，故 $d(q,m) \geq \max\{0, d(q,c_j) - r_j\}$；对 m 取下确界即得 $s_M(q) \geq L(q)$。▌